GST Helper for E-Commerce — Rebuild Kit

> One file that explains the software, how to run it, how to use it, and the **copy-paste prompts** that make Claude build the same software again — with a new look and feel.

Live app (original)	https://gst-helper-t4ug.onrender.com/
Source code (original)	https://github.com/ayushtomarpsl-commits/GST-Filing-E-Com
Built with	Python 3.12 · Flask · pandas · openpyxl · waitress
Built by	Claude (Claude Code), for Ayush Tomar
Kit version	1.0 — 05-Nov-2026

Note: the live app is on Render's free plan. If nobody used it for 15 minutes, the first visit takes about 50 seconds to wake up. That is normal.

Contents

1. What the software does
2. How to rebuild it with Claude — step by step
3. The MASTER PROMPT (copy-paste this)
4. Follow-up prompts
5. Running guide — local and hosted
6. User guide — monthly filing, step by step
7. Acceptance checklist — is the rebuild correct?
8. Lessons from real filings (history)
9. Reference data

---

1. What the software does

Indian sellers on **Amazon, Flipkart and Meesho** must file **GSTR-1** (sales return) and **GSTR-3B** (summary return + tax payment) every month on the GST portal. Each marketplace gives different

reports, with different columns, sheet names, state spellings and sign conventions. Joining them by hand is slow and error-prone.

GST Helper does it for them:

1. The seller downloads the monthly reports from each marketplace (and the GSTR-2B JSON from the GST portal) into **one folder**. Zips can stay zipped.
2. The seller points the app at that folder (local mode) or uploads it from the browser (hosted mode).
3. The app **auto-detects** every file by its content (sheets / columns — not the file name), finds the return month, and **excludes wrong-month files**.
4. It parses, nets sales against returns and credit notes, and consolidates all platforms state-wise.
5. It produces **portal-ready JSON** files: - **GSTR-1**: Table 7 B2CS, Table 5 B2CL, Table 12 HSN (B2C), Table 13 Documents issued — one combined JSON for *Prepare Offline* upload, plus one JSON per table. - **GSTR-3B**: Table 3.1(a), 3.1(d), 3.2, Table 4 ITC (from GSTR-2B), Table 5. - Extras: a B2CS CSV for the offline tool and an Excel reconciliation workbook.
6. It **validates against GST portal rules before upload** and shows plain-English red / yellow / blue messages: what is wrong, what is missing, which file to download and from where.

Core design rule (learned from real filings): never drop data silently. Anything that cannot be processed becomes a visible error or warning.

Screens

Page	What is on it
Home (`/`)	Folder path box (local mode only) · "Browse folder" upload with file list and progress bar · optional month box (MMYYYY) · manual file upload · "What to download" cards for the 4 sources
Results (`/process`)	Period + seller GSTIN · guidance banners (errors → warnings → info) · status tiles per source (ok / partial / error / missing) · totals strip (taxable, IGST, CGST, SGST, cess, total tax) · one card per GSTR-1 table (stats, data preview, **Copy JSON**, **Download**) · big "Complete GSTR-1 JSON" card · GSTR-3B card listing every portal field · extra downloads (CSV, Excel) · file-scan table (every file, why it was used or skipped)
Help (`/help`)	5-step picture guide (inline SVG click-paths for each website) + FAQ
Guided tour	First-visit spotlight tour on the home page; "Show guide" button in the header replays it

2. How to rebuild it with Claude — step by step

You need: a computer with **Python 3.11+**, and **Claude Code** (desktop app, CLI, or IDE extension). Claude.ai chat also works, but Claude Code is better because it can create the files and run the tests itself.

1. **Make an empty folder**, for example `D:\gst-helper-v2`.
2. **Open Claude Code in that folder.**
3. **Decide the new look.** In the master prompt, find the block `DESIGN BRIEF` and replace the example with your own style (colours, mood, font, layout). If you leave it, Claude will pick a fresh style that is clearly different from the original navy/white look.
4. **Copy the whole MASTER PROMPT** from section 3 (everything inside the box) and paste it into Claude as one message.
5. **Let Claude work.** It builds in phases (engine → tests → web UI → help and tour → deploy files) and runs the tests after each phase. If it asks a question, answer it. If it stops halfway (long jobs can hit limits), send **Follow-up prompt A** ("continue").
6. **Run the app** (section 5): double-click `run.bat`, or run `python web_app.py`. The browser opens at http://127.0.0.1:5000.
7. **Check it** with the acceptance checklist (section 7). Claude creates synthetic sample reports in `samples/` so you can test without real data.
8. **Test with one real month.** Compare the app's numbers with a month that was already filed on the portal. Use **Follow-up prompt C** if numbers differ.
9. **Host it** (optional) with **Follow-up prompt E** and section 5.2.

> Tip: if the prompt is too long for your plan, use the **phased prompts** in section 4 (Follow-up prompt F). Paste the master prompt first with the line "Only do Phase 1 now", then ask for each next phase.

3. The MASTER PROMPT (copy-paste this)

Copy everything between the two lines of `~~~~`.

Then paste it into Claude Code in an empty folder.

```
You are building "GST Helper for E-Commerce": a web app that turns Indian
marketplace seller tax reports (Amazon, Flipkart, Meesho) plus the GST-portal
GSTR-2B JSON into GST-portal-ready GSTR-1 and GSTR-3B JSON files.

A working original exists:
  Live:   https://gst-helper-t4ug.onrender.com/
  Source: https://github.com/ayushtomarpsl-commits/GST-Filing-E-Com
Rebuild the SAME behaviour and business logic (every rule below), but with a
NEW, clearly different visual design (see DESIGN BRIEF). You may read the
original source for reference if you can reach it; this spec is the contract.

================================================================
0. WORKING RULES
================================================================
- Build in phases, in this order, and run the tests at the end of each phase:
  Phase 1 engine (detection, parsers, consolidation, JSON builders, rules)
  Phase 2 synthetic sample files + tests
  Phase 3 web app (Flask) + results dashboard
  Phase 4 help page + guided tour
  Phase 5 hosting files (render.yaml, run.bat, README, DEPLOY_GUIDE)
- Create CLAUDE.md (short rules + pointer) and docs/PROJECT-CONTEXT.md
  (what we build, decisions, status, next steps, change log newest first)
  at the start, and update them after each phase.
- Use simple, clear English in all UI text. Commands for Windows users must
  be PowerShell 5.1 syntax (no "&&").
- NEVER DROP DATA SILENTLY. Any row, file, state, section or document type
  that cannot be processed must become a visible error or warning.
- Never commit real business data. .gitignore must block: .jobs/, real
  report folders, *.zip, generated gstr1_*.json / gstr3b_*.json, *errorReport*.json.
- Money: round to 2 decimals. In JSON write a number as an int when it is
  whole (5 not 5.0), else a float with max 2 decimals. Compact JSON
  (separators ",", ":"), UTF-8, ensure_ascii False.
- Use type hints and dataclasses. Keep modules small and focused.

________________________________________________________________
```

4. Follow-up prompts

Paste these after the master prompt, when needed.

A — Continue after a stop

```
Continue building from where you stopped. First read CLAUDE.md and
```

```
docs/PROJECT-CONTEXT.md, run the tests, then go on with the next unfinished phase.
```

B — Change only the look

```
Keep all logic, routes, element ids and tests exactly as they are. Redesign only
templates/*.html and static/css/*.css to this style: <describe your style>.
Then run the web tests and show me screenshots of the home, results and help pages
using the synthetic samples.
```

C — Numbers differ from a month already filed

```
I processed a real month. The app shows <value> for <table / state / HSN>, but
the portal (or my CA) shows <value>. Do not guess. Find which file and which rows
make the difference, explain it in simple English, and fix the logic only if the
app is wrong. Add a regression test for this case. Never print or store my real
GSTIN, names or amounts in code, tests or docs.
```

D — A platform changed its report format

```
<Platform> changed its report. Here are the new sheet names and column headers:
<paste headers only, no customer data>. Update detection and the parser, keep old
formats working, make missing columns fail loudly, and add a synthetic sample + test.
```

E — Put it on the internet (Render, free)

```
Prepare this app for public hosting on Render with PUBLIC_MODE=1. Check .gitignore
blocks all real data, show me `git status`, then give me the exact steps
(PowerShell 5.1 syntax) to push to a NEW PRIVATE GitHub repo and deploy with a
Render Blueprint. Do not push anything without my OK.
```

F — Phased mode (for smaller plans)

```
Use the master prompt above as the full spec. Only do Phase <N> now. Run the tests
for that phase, update docs/PROJECT-CONTEXT.md, and stop.
```

G — Add a new platform (example: Myntra / JioMart / Shopsy)

```
Add <platform> support following the same pattern: content fingerprint in
gst_autodetect.py, parser producing TaxRecord + HsnRow + DocEntry, status tile,
```

```
download help text, help-page block, synthetic sample and tests. Here are the
report sheet names and headers: <paste headers only>.
```

---

5. Running guide — local and hosted

5.1 Run on your own computer (Windows)

1. Install **Python 3.12** from python.org (tick *Add python.exe to PATH*).
2. Open the project folder.
3. **Easiest:** double-click `run.bat`. It installs packages and starts the app.
4. **Or** in PowerShell:

```
pip install -r requirements.txt
```

```
python web_app.py
```

Then the browser opens at **http://127.0.0.1:5000**. Keep the window open while you use the app. Press **Ctrl+C** to stop.

Options: `python web_app.py --port=5050` (other port), `--no-browser`, `--public` (test public mode locally).

In local mode the reports never leave your computer, and you can paste a folder path such as `C:\Users\you\Downloads\gst june`.

5.2 Host it on the internet (Render free plan)

1. Make sure `.gitignore` blocks your real data. Run `git status` and check that no report files, zips or generated JSONs are listed.
2. Push the project to a **private** GitHub repository.
3. On https://render.com sign up with GitHub → **New +** → **Blueprint** → pick the repo. Render reads `render.yaml` and creates the service `gst-helper`.
4. Click **Deploy**. After 3–5 minutes the site is live at `https://<name>.onrender.com`.
5. Every `git push` to `main` redeploys automatically.

Public mode (set by `render.yaml`): no folder-path box, upload only, files deleted within 1 hour. Free-plan limit: the service sleeps after 15 minutes idle; the next visit waits about 50 seconds.

Alternative: PythonAnywhere (free, never sleeps, renew every 3 months) — see `DEPLOY_GUIDE.md`.

5.3 Run the tests

```
python test_portal_rules.py
```

```
python test_gstr3b.py
```

```
python test_web_modes.py "<folder with sample reports>"
```

```
python test_ground_truth.py "<folder of a month already filed>"
```

(In a rebuild, Claude may put tests in `tests/` and use `pytest` instead.)

---

6. User guide — monthly filing, step by step

Order every month: download reports → one folder → process → upload GSTR-1 → file GSTR-1 → check GSTR-3B → pay and file GSTR-3B.

Step 1 — Download the reports (up to 7 files)

Source	Where	File	Fills
Amazon	Seller Central → GST Reports → *GST Ready-to-File Reports* → Download	`.xlsx`	B2CS, B2CL, HSN
Amazon	Seller Central → Reports → Tax Document Library → *MTR B2C*	`.csv`	Table 13 invoice numbers
Flipkart	Seller Hub → Reports → GST Reports → *GSTR return report*	`.xlsx`	B2CS, HSN, invoice series
Flipkart	Seller Hub → Reports → *Sales Report*	`.xlsx`	Credit-note series

Source	Where	File	Fills
Meesho	Supplier Panel → Download → *GST Report*	`.zip`	B2CS, HSN
Meesho	Supplier Panel → Download → *Supplier Tax Invoice*	`.zip`	Table 13 series
GST portal	Returns Dashboard → month → GSTR-2B → Download → *GENERATE JSON FILE TO DOWNLOAD*	`.json`	GSTR-3B Table 4 (ITC)

Menu names can change a little over time.

Step 2 — Put everything in ONE folder

- One folder per month. Do not mix months (wrong-month files are excluded automatically, with a warning).
- Zips stay zipped. Keep the original file names.

Step 3 — Process

- **Hosted:** open https://gst-helper-t4ug.onrender.com/ → **Browse folder** → pick the folder → **Upload & process**.
- **Local:** paste the folder path → **Process folder**.
- Leave the month empty (auto-detected) or type MMYYYY, e.g. `062026`.
- Read the results page top to bottom:
  - **Red** = must fix, then process again.
  - **Yellow** = read and decide; files were still made.
  - **Blue** = information (e.g. missing optional file, TCS amount).
  - **File scan** table = why each file was used, skipped or excluded.

Step 4 — Upload GSTR-1

1. Log in at https://services.gst.gov.in.
2. Returns Dashboard → month → **GSTR-1** → **PREPARE OFFLINE**.
3. Upload → Choose File → `gstr1_full_<MMYYYY>.json` (from the *Complete GSTR-1 JSON* card).
4. Wait a few minutes. If the status is *Processed with Error*, download the error report JSON, drop it in the same folder and process again — the app explains each error and says whether it is fixed.
5. Open the GSTR-1 online view and compare Tables 5, 7, 12, 13 with the app's cards.
6. **Generate Summary** → **Submit** → **File** with EVC or DSC.

Uploading replaces the matching sections already on the portal. Filing is final.

Step 5 — GSTR-3B (after GSTR-1 is filed)

1. The portal auto-drafts GSTR-3B from your GSTR-1 and GSTR-2B.
2. Compare it with the app's **GSTR-3B card**: 3.1(a), 3.2 state rows, Table 4.
3. Or upload `gstr3b_<MMYYYY>.json` through the GSTR-3B offline utility.
4. Accept marketplace TCS: **Services** → **Returns** → **TDS and TCS credit received** (the app shows the approximate amount).
5. No GSTR-2B JSON given? The app leaves Table 4 out on purpose — keep the portal's auto-drafted ITC.
6. Pay any tax due and file.

> This tool prepares data to help you. You are responsible for what you file. When in doubt, ask your tax advisor.

---

7. Acceptance checklist — is the rebuild correct?

Tick every line before using the rebuild for a real filing.

Detection

- ☐ A folder with all 7 sample files: every file shows the right platform and kind in *File scan*.
- ☐ Zips inside zips are opened; PDFs inside zips show as one "N PDF invoice copies" line.
- ☐ A file from another month is shown **excluded** with the month message.
- ☐ Two copies of the same report: one used, the other marked duplicate.

GSTR-1

- ☐ B2CS: one row per (state, rate, INTRA/INTER); tax = taxable × rate exactly; own state = CGST+SGST equal halves.
- ☐ "DAMAN & DIU" and "CHATTISGARH" rows map to POS 26 and 22 — no error.
- ☐ An Amazon B2C Large invoice appears in Table 5 and **not** in B2CS.
- ☐ HSN rows are merged across platforms on HSN + UQC + rate; promo gross-tax row is corrected **with a warning**.
- ☐ Table 13: Amazon/Flipkart contiguous runs; Meesho one span with gaps counted as cancelled.

- [ ] The full JSON has `gstin`, `fp`, `version: GST3.2.4`, `hash: hash` and only non-empty sections.

GSTR-3B

- [ ] 3.1(a) equals the GSTR-1 B2CS + B2CL totals.
- [ ] 3.2 lists only other states (never the seller's own state).
- [ ] Without GSTR-2B: `itc_elg` is **absent**, card says "add GSTR-2B".
- [ ] With GSTR-2B of another GSTIN or month: red error, ITC not used.
- [ ] A 2B with an `impg` section gives a red "fill manually" error.

Web

- [ ] Local mode: folder path works; browser opens automatically.
- [ ] Public mode: no folder-path box; a posted `folder_path` is ignored by the server.
- [ ] Upload shows progress; an over-200 MB selection is blocked with a clear message.
- [ ] `/download_file/../../etc/passwd`-style paths return 404.
- [ ] Copy JSON and Download work on every card.
- [ ] Job folders older than 1 hour are deleted.
- [ ] Help page and guided tour work; "Show guide" replays the tour.

Real-data check

- [ ] One already-filed month gives the same totals as the portal (within ₹1).

---

8. Lessons from real filings (history)

These came from using the original on real months (Aug–Oct 2026). The master prompt already includes them; they are listed here so a human understands *why*.

Problem seen	Cause	What the app does now
Portal rejected a row with **RET191240 "Invalid POS selected"**	Meesho wrote "DAMAN & DIU"; old code 25 was retired on 26-Jan-2020	All Daman/Dadra spellings → 26; retired 25→26, 28→37; pre-upload check blocks POS 25/28
"Could not resolve Meesho state POS code for 'MADHYA PRADESH'"	Spelling not in the name map	Big name map with variants (CHATTISGARH, TELENGANA, PUDDUCHERRY, NCT OF DELHI…); unknown states still fail loudly

Problem seen	Cause	What the app does now
"Table 3.2 totals exceed Table 3.1(a)"	Home state was net negative (returns > sales), pulling 3.1(a) below 3.2	Taxable check skipped in that case; IGST check stays hard
Negative B2CS row for a state	Returns more than sales that month	Warning only. The portal accepted it in practice. Options: amend the old month or carry forward — the CA decides; 3B must match the GSTR-1 finally filed
HSN tax bigger than taxable × rate	Amazon counted gross tax on free promo shipments	Tax recomputed from taxable value, with a visible warning
Old April file in a June folder	Mixed months	Wrong-month files excluded automatically
"Meesho invoice series missing" though a Meesho invoice zip was in the folder	It was the logistics bill zip, not the Supplier Tax Invoice	Explain the difference in the warning
Marketplace GSTIN failed the check-digit test	Operator GSTINs printed in reports fail the public algorithm	Format check only for marketplace GSTINs; seller GSTIN gets a warning, not an error
Portal echoed `rt` 5 where the JSON had 5.0	Number vs text compare	Error-report rows compared as numbers
"Do we need to rebuild the app?"	Code change	No — plain Python: stop and start again; press Ctrl+F5 for stale pages

Known open items in the original (good first tasks for a rebuild):

- `test_ground_truth.py` crashes with `KeyError: 5` in the Table 13 step on the sample folder.
- No option yet to carry a negative state row forward automatically.
- The app does not yet name the Meesho logistics-invoice zip when it sees it.
- Legacy `app.py` (Streamlit) and `flipkart_gstr1_b2cs.py` are still in the repo.

9. Reference data

State codes (valid POS): 01 Jammu & Kashmir · 02 Himachal Pradesh · 03 Punjab · 04 Chandigarh · 05 Uttarakhand · 06 Haryana · 07 Delhi · 08 Rajasthan · 09 Uttar Pradesh · 10 Bihar · 11 Sikkim · 12 Arunachal Pradesh · 13 Nagaland · 14 Manipur · 15 Mizoram · 16 Tripura · 17 Meghalaya · 18 Assam · 19 West Bengal · 20 Jharkhand · 21 Odisha · 22 Chhattisgarh · 23 Madhya Pradesh · 24 Gujarat · 26 Dadra & Nagar Haveli & Daman & Diu · 27 Maharashtra · 29 Karnataka · 30 Goa · 31 Lakshadweep · 32 Kerala · 33 Tamil Nadu · 34 Puducherry · 35 Andaman & Nicobar Islands · 36 Telangana · 37 Andhra Pradesh · 38 Ladakh · 97 Other Territory. **Retired:** 25 → 26, 28 → 37 (portal error RET191240).

GST rates accepted: 0, 0.1, 0.25, 1, 1.5, 3, 5, 6, 7.5, 12, 18, 28, 40.

Table 13 doc_num: 1 = invoices for outward supply, 4 = debit notes, 5 = credit notes.

Marketplace operator GSTINs used as defaults: Amazon 27AAPCA8805L1ZS · Flipkart 29AAACF1435D1Z5 · Meesho 09AARCM9332R1CM.

Original project files (for anyone reading the source):

File	Job
`web_app.py`	Flask server, public mode, job folders, downloads
`gst_autodetect.py`	Folder/zip scanner, content fingerprinting, period hints
`gst_parsers.py`	State normalisation, B2CS parsers, consolidation, Excel
`gst_ext_parsers.py`	HSN, document series, B2CL parsers; HSN tax reconciliation
`gst_builder.py`	Full pipeline, validation, guidance, result cards
`gst3b_builder.py`	GSTR-2B parser, GSTR-3B builder and checks
`gst_portal_rules.py`	Portal master data and pre-upload rules
`gst_processor.py`	Offline-tool CSV writer (+ legacy single-file pipeline)
`templates/` , `static/`	Home, results, help pages; CSS; upload + tour JS
`test_*.py`	Ground truth, GSTR-3B, portal rules, web modes
`app.py` , `flipkart_gstr1_b2cs.py`	Legacy Streamlit UI and Flipkart-only CLI (not needed in a rebuild)